

Bases de données relationnelles

Modèle, langages, optimisation

v1.9

B. Amann, C. Crochepeyre, M. Crucianu, M. Ferecatu,
D. Gross-Amblard, P. Rigaux, V. Thion, N. Travers, D. Vodislav,
V. Sans, M. Scholl

Plan du cours

- 1 Concurrency et reprise après panne

Plan du cours

- 1 Concurrency et reprise après panne

Plan du cours

- 1 Concurrence et reprise après panne
 - Introduction et problématique
 - Contrôle de concurrence
 - Tolérance aux pannes
 - Transactions dans les SGBD relationnels

1. La notion de transaction

Modèle de base de données

- BD centralisée, accès concurrent de **plusieurs** programmes
- modèle simplifié
 - ▶ BD = ensemble d'*enregistrements* nommés
Ex. x: 3 ; y: "Toto" ; z: 3.14 ...
 - ▶ *opérations* sur les enregistrements: lecture, écriture, création

Programmes et transactions

- exécution d'un programme accédant à la BD → séquence d'opérations sur les enregistrements
- opérations : lecture ($val = Read(x)$), écriture ($Write(x, val)$)
- découpage en *transactions*

Transaction

- opérations de contrôle de transaction : Start (démarrage), Commit (validation), Abort (annulation)
- transaction = séquence d'opérations qui démarre par *Start* et se termine par *Commit* ou par *Abort*
- cohérence logique (maxi-opération), unité d'annulation

Exemple: programme de crédit d'un compte bancaire

Crédit (*Enreg* compte; *Val* montant)

Val temp;

begin *Start*;

temp = *Read*(compte);

Write(compte, temp+montant);

Commit;

end;

- les entrées du programme: le compte (enregistrement) et le montant du crédit (valeur)
- l'exécution du programme → transaction
- plusieurs exécutions concurrentes du même programme possibles

Exemple: transfert entre deux comptes

Transfert (*Enreg* source, dest; *Val* montant)

Val temp;

begin *Start*;

temp = *Read*(source);

if temp < montant then *Abort*;

else *Write*(source, temp-montant);

temp = *Read*(dest);

Write(dest, temp+montant); *Commit*;

end if;

end;

- l'exécution peut produire des transactions différentes:

1. *Start*, *Read*(source), *Abort*

2. *Start*, *Read*(source), *Write*(source), *Read*(dest), *Write*(dest),
Commit

Mise-à-jour de la BD

Deux variantes:

- *immédiate*: modification immédiate de la BD, visible par les autres transactions
- *différée*: chaque transaction travaille sur des copies, avec mise-à-jour de la BD à la fin de la transaction

Hypothèse: mise-à-jour immédiate

Propriétés des transactions (ACID)

Atomicité: une transaction doit s'exécuter en totalité, une exécution partielle est inacceptable

- une transaction interrompue doit être annulée (Abort)

Cohérence: respect des contraintes d'intégrité sur les données

- $\text{solde}(\text{compte}) \geq 0$; $\text{solde}(\text{source}) + \text{solde}(\text{dest}) = \text{const}$
- une transaction modifie la BD d'un état initial cohérent à un état final cohérent
- pendant la transaction, l'état peut être incohérent!

Isolation: une transaction ne voit pas les effets des autres transactions en cours d'exécution

- la transaction s'exécute comme si elle était seule
- objectif: exécution concurrente des transactions équivalente à une exécution en série (non-concurrente)

Durabilité: les effets d'une transaction validée par Commit sont permanents

- on ne doit pas annuler une transaction validée

Concurrence

- plusieurs transactions s'exécutent en même temps
- le système exécute les opérations en séquence!
- concurrence = entrelacement des opérations de plusieurs transactions

Notation

- transaction T_i = séquence d'opérations
- opérations $r_i[x]$, $w_i[x]$, a_i , c_i
- remarque: les valeurs lues ou écrites ne comptent pas ici!
- la séquence se termine par a_i ou c_i , qui n'apparaissent jamais ensemble

Exemple :

T_1 : $r_1[x]$ $w_1[x]$ c_1 (crédit)

T_2 : $r_2[x]$ $w_2[x]$ $r_2[y]$ $w_2[y]$ c_2 (transfert)

T_3 : $r_3[x]$ a_3 (transfert impossible)

Exécution concurrente (histoire)

- séquence d'opérations de plusieurs transactions
- entrelacement des opérations des transactions
- histoire complète: les transactions sont entières

Exemple :

H_1 : $r_1[x]$ $r_2[x]$ $w_2[x]$ $r_2[y]$ $w_1[x]$ $w_2[y]$ c_2 c_1 (crédit + transfert, histoire complète)

H_2 : $r_1[x]$ $r_2[x]$ $w_2[x]$ $r_2[y]$ (histoire incomplète)

2. Contrôle de concurrence

Objectifs

- concurrence = entrelacement des opérations des transactions
- concurrence parfaite: toute opération est exécutée dès son arrivée dans le système
- problème: tout entrelacement n'est pas acceptable
- objectif: exécution correcte en respectant les propriétés ACID

Problèmes de concurrence

1. Perte d'une mise à jour

Ex. $T_1: \text{Crédit}(x, 100) = r_1[x] \ w_1[x] \ c_1$ $T_2: \text{Crédit}(x, 50) = r_2[x] \ w_2[x] \ c_2$
 au début, $x=200$

- $H = \underline{r_1[x]_{x:200}} \ r_2[x]_{x:200} \ \underline{w_1[x]_{x:300}} \ w_2[x]_{x:250} \ \underline{c_1} \ c_2$
- Résultat: $x=250$ au lieu de $x=350$ ($w_1[x]$ est perdu car la lecture dans T_2 n'en tient pas compte)
- Problème de cohérence à cause d'une **lecture non-répétable**
- Une fois une donnée lue, une transaction devrait retrouver la même valeur plus tard
- Ici T_2 lit $x=200$, mais après $w_1[x]$, x devient 300

2. Dépendances non-validées

Ex. Les mêmes transactions, mais T_1 est annulée

- $H = \underline{r_1[x]_{x:200}} \underline{w_1[x]_{x:300}} r_2[x]_{x:300} w_2[x]_{x:350} c_2 \underline{a_1}$
- $r_2[x]$ utilise la valeur non-validée de x écrite par $w_1[x]$
- L'annulation de T_1 impose l'annulation de T_2 , qui est validée!
- Problème de durabilité à cause de la **lecture d'une valeur non-validée** ("lecture sale")

3. Analyse incohérente

Ex. $T_1 =$ transfert $x \rightarrow y$ de 50; $T_2 =$ calcul dans z de la somme $x + y$
au début, $x=200$, $y=100$

- $T_1 = r_1[x] \ w_1[x] \ r_1[y] \ w_1[y] \ c_1$; $T_2 = r_2[x] \ r_2[y] \ w_2[z] \ c_2$
- $H = \underline{r_1[x]_{x:200} \ w_1[x]_{x:150} \ r_2[x]_{x:150} \ r_2[y]_{y:100} \ w_2[z]_{z:250} \ c_2 \ r_1[y]_{y:100} \ w_1[y]_{x:150} \ c_1}$
- Résultat: $z=250$ au lieu de $z=300$ ($r_2[x]$ est influencé par T_1 , mais $r_2[y]$ non)
- Problème de cohérence à cause de la **lecture d'une valeur non-validée** ("lecture sale")

4. Objets fantômes

- similaire à l'analyse incohérente, mais produit par la création/suppression d'enregistrements (**objets fantômes**)
- traité plus loin dans le cours

5. Ecritures incohérentes

Ex. Ecriture d'un même solde dans deux comptes

T_1 = écriture de 100 dans x et y ; T_2 = écriture de 200 dans x et y

- $T_1 = w_1[x] w_1[y] c_1$; $T_2 = w_2[x] w_2[y] c_2$
- $H = \underline{w_1[x]_{x:100} w_2[x]_{x:200} w_2[y]_{y:200} w_1[y]_{y:100}} \underline{c_1 c_2}$
- Résultat: $x \neq y$! ($x = 200, y = 100$)
- Problème de cohérence à cause de l'**écriture sur des données non-validées**

Contrôle de concurrence

- solution: algorithmes de réordonnancement des opérations
- critère de correction utilisé: exécution sérialisable
(équivalente à une exécution en série *quelconque* des transactions)

Remarques

- réordonner $\Rightarrow$ retarder certaines opérations
- objectif : un maximum de concurrence, donc un minimum de retards
- *l'ordre des opérations dans chaque transaction doit être respecté !*

3. Le problème des annulations

Annuler dans la BD les effets d'une transaction $T \iff$

- annuler les effets de T (les écritures)
- annuler les transactions qui utilisent les écritures de T (dépendances non-validées)

Conclusion: une transaction validée risque encore d'être annulée

Types d'exécutions concurrentes par rapport à l'annulation

- exécution recouvrable
- exécution qui évite les annulations en cascade
- exécution stricte

Exécution recouvrable = permet de ne pas avoir à annuler une transaction validée

Ex. $w_1[x] \ r_2[x] \ w_2[y] \ c_2 \ a_1$ (a_1 oblige l'annulation de T_2 , qui est validée)

Définitions:

- T_2 lit x de T_1 : T_2 lit la valeur écrite par T_1 dans x
- T_2 lit de T_1 : T_2 lit au moins un enregistrement de T_1

Solution: si T_2 lit de T_1 , alors T_2 doit valider après T_1

$\Rightarrow$ *retardement des Commit*

(dans l'exemple, retardement de c_2 après la fin de T_1)

Éviter les annulations en cascade

- exemple précédent: même si c_2 est retardé, T_2 sera quand même annulée à cause de $T_1 \rightarrow$ annulation en cascade

Ex. $\underline{w_1[x]} \ r_2[x] \ w_2[y] \ \underline{a_1}$ (a_1 oblige a_2)

- l'annulation d'une transaction, même non validée, est gênante
- *solution*: T_2 ne doit lire qu'à partir de transactions validées

$\Rightarrow$ *retardement des lectures*

(dans l'exemple, retardement de $r_2[x]$ après la fin de T_1)

Exécution stricte

= évite les annulations en cascade + annulation simple des écritures

- l'annulation des écritures: par restauration des images avant
- image avant de x par rapport à $w[x]$ = valeur de x avant l'écriture

Problèmes de restauration des images avant

Ex. $w_1[x, 2]$ $w_2[x, 3]$ a_1 a_2 (au début $x=1$)

image avant($w_1[x]$)=1, image avant($w_2[x]$)=2

a_1 restaure $x=1$: erreur, ne tient pas compte de $w_2[x, 3]$

a_2 restaure $x=2$: erreur, cette valeur est déjà annulée

- *solution*: $w_2[x]$ attend la fin de tout T_i qui a écrit x

⇒ *retardement lectures + écritures*

(dans l'exemple, retardement de $w_2[x]$ après la fin de T_1)

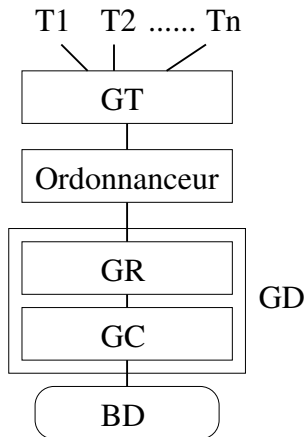
4. Tolérance aux pannes

Mémoire SGBD = Mémoire stable (disque) + Mémoire volatile

Catégories de pannes

- **de transaction**: annulation de la transaction
- **de système**: perte du contenu de la mémoire volatile
 - ▶ opération *Restart*: restaurer l'état cohérent de la BD avant la panne
 - ▶ journalisation
- **de support physique**: perte contenu mémoire stable
 - ▶ duplication par mirroring, archivage

5. Modèle abstrait de la BD



BD centralisée, transactions concurrentes:

Composantes

- **Gestionnaire de transactions (GT)**: reçoit les transactions et les prépare pour exécution
- **Ordonnanceur (Scheduler)**: contrôle l'ordre d'exécution des opérations (séquences sérialisables et recouvrables)
- **Gestionnaire de reprise (GR)**: Commit + Abort
- **Gestionnaire du Cache (GC)**: gestion de la mémoire volatile et de la mémoire stable

$GR + GC = GD$ (**Gestionnaire de données**): assure la tolérance aux pannes

Plan du cours

1 Concurrence et reprise après panne

- Introduction et problématique
- **Contrôle de concurrence**
- Tolérance aux pannes
- Transactions dans les SGBD relationnels

1. Théorie de la sérialisabilité

Objectif du contrôle de concurrence

= produire une exécution *sérialisable* des transactions

Exécution sérialisable : équivalente à une exécution en série *quelconque* des transactions

Exécution en série

- transactions exécutées l'une après l'autre (aucun entrelacement)

Ex. au début $x=200$; $T_1 = \text{crédit } x \text{ de } 100$; $T_2 = \text{crédit } x \text{ de } 50$

$H_1 = T_1 T_2 = r_1[x]_{x:200} w_1[x]_{x:300} C_1 r_2[x]_{x:300} w_2[x]_{x:350} C_2$

$H_2 = T_2 T_1 = r_2[x]_{x:200} w_2[x]_{x:250} C_2 r_1[x]_{x:250} w_1[x]_{x:350} C_1$

Équivalence de deux exécutions (histoires)

- 1 Avoir les mêmes transactions et les mêmes opérations
- 2 Produire le même effet sur la BD (écritures)
- 3 Produire le même effet dans les transactions (lectures)

Ex. $H_1 \neq H_2$ (conditions 1 et 2 respectées, mais pas 3)

Conflit

Def. $p_i[x]$ et $q_j[y]$ sont en conflit $\iff$

- $i \neq j, x=y$ (transactions différentes, même enregistrement)
- $p_i[x] \ q_j[x]$ n'a pas le même effet que $q_j[x] \ p_i[x]$

Remarques

- conflit = l'inverse de la commutativité
- commutativité = même effet sur la BD et sur les transactions
- conflits: $w_i[x]-w_j[x], r_i[x]-w_j[x], w_i[x]-r_j[x]$
- seul le couple $r_i[x]-r_j[x]$ n'est pas en conflit

Critère d'équivalence basé sur les conflits

- Avoir les mêmes transactions et les mêmes opérations
- *Avoir le même ordre des opérations conflictuelles dans les transactions non-annulées*

Ce dernier critère:

- couvre les conditions 2 et 3 de la définition
- est plus strict, mais plus facile à vérifier que 2 et 3

Ex. $H_1 \not\equiv H_2$ (H_1 : $r_1[x] - w_2[x]$; H_2 : $w_2[x] - r_1[x]$)

Exemple

$T_1: r_1[x] \ w_1[y] \ w_1[x] \ c_1$ $T_2: w_2[x] \ r_2[y] \ w_2[y] \ c_2$

$H_1: \frac{r_1[x] \ w_2[x] \ w_1[y] \ r_2[y] \ w_1[x] \ w_2[y] \ c_1 \ c_2}{\text{conflicts: } r_1[x] - w_2[x], w_2[x] - w_1[x], w_1[y] - r_2[y], w_1[y] - w_2[y]}$

$H_2: \frac{r_1[x] \ w_1[y] \ w_2[x] \ w_1[x] \ c_1 \ r_2[y] \ w_2[y] \ c_2}{\text{conflicts: } r_1[x] - w_2[x], w_2[x] - w_1[x], w_1[y] - r_2[y], w_1[y] - w_2[y]}$

$\implies H_2 \equiv H_1$

$H_3: w_2[x] \ r_2[y] \ \frac{r_1[x] \ w_2[y] \ w_1[y] \ c_2 \ w_1[x] \ c_1}{\text{conflicts: } w_2[x] - r_1[x], w_2[x] - w_1[x], r_2[y] - w_1[y], w_2[y] - w_1[y]}$

$\implies H_3 \not\equiv H_1$

$H_4: w_2[x] \ r_2[y] \ w_2[y] \ c_2 \ \frac{r_1[x] \ w_1[y] \ w_1[x] \ c_1}{\text{conflicts: } w_2[x] - r_1[x], w_2[x] - w_1[x], r_2[y] - w_1[y], w_2[y] - w_1[y]}$ (histoire sériale)

$\implies H_4 \equiv H_3 \implies H_3$ sérialisable

Théorème de sérialisabilité

Graphe de sérialisation d'une exécution H : $SG(H)$

- *noeuds*: transactions T_i validées dans H
- *arcs*: si p et q conflictuelles, $p \in T_i$, $q \in T_j$, p avant q
 $\Rightarrow$ arc $T_i \rightarrow T_j$

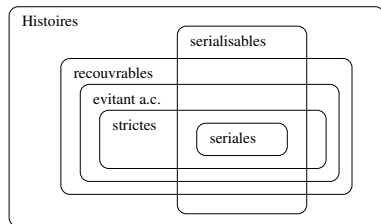
Théorème: H sérialisable $\iff SG(H)$ acyclique

$$H_1, H_2: T_1 \xrightarrow{\quad} T_2$$

$$H_3, H_4: T_1 \longleftarrow T_2$$

Propriétés de recouvrabilité

- à tout moment une transaction peut être annulée (panne)
- propriétés: stricte $\Rightarrow$ pas d'annulation en cascade $\Rightarrow$ recouvrable
- *la sérialisabilité* dépend de l'ordre des opérations
la recouvrabilité dépend de l'ordre des Commit/Abort
 $\Rightarrow$ les deux propriétés sont orthogonales



Exemple

$T_1: r_1[x] \ w_1[y] \ r_1[z] \ w_1[z] \ c_1$ $T_2: r_2[x] \ w_2[y] \ r_2[z] \ w_2[z] \ c_2$

H: $r_1[x]$ $r_2[x]$ $w_1[y]$ $r_1[z]$ $w_1[z]$ $w_2[y]$ $r_2[z]$ $w_2[z]$

conflicts: $w_1[y] - w_2[y]$, $r_1[z] - w_2[z]$, $w_1[z] - r_2[z]$, $w_1[z] - w_2[z]$

$\Rightarrow$ H est sérialisable pour n'importe quelle position de c_1 et de c_2

$H_1: r_1[x] \ r_2[x] \ w_1[y] \ r_1[z] \ w_1[z] \ w_2[y] \ r_2[z] \ w_2[z] \ c_2 \ c_1$

H_1 n'est pas recouvrable (T_2 lit z de T_1 et c_2 après c_1)

$H_2: r_1[x] \ r_2[x] \ w_1[y] \ r_1[z] \ w_1[z] \ w_2[y] \ r_2[z] \ c_1 \ w_2[z] \ c_2$

H_2 n'évite pas les annulations en cascade (T_2 lit z de T_1 avant c_1)

$H_3: r_1[x] \ r_2[x] \ w_1[y] \ r_1[z] \ w_1[z] \ w_2[y] \ c_1 \ r_2[z] \ w_2[z] \ c_2$

H_3 n'est pas stricte (T_1 écrit y et ensuite T_2 écrit y avant c_1)

Remarques

- *Équivalence*: la définition basée sur les conflits est suffisante et facile à utiliser, mais pas nécessaire

Ex. $H_1: w_1[x] w_2[x] w_3[x]$ $H_2: w_2[x] w_1[x] w_3[x]$

$H_1 \not\equiv H_2$ dans le sens des conflits $H_1 \equiv H_2$ dans le sens général

- *Conflit* = l'inverse de la commutativité
 $\implies$ extensible à d'autres opérations que Read et Write

	Read	Write	Incr	Decr
Read	oui	non	non	non
Write	non	non	non	non
Incr	non	non	oui	oui
Decr	non	non	oui	oui

2. Contrôle par verrouillage à deux phases

Principe

- stratégie pessimiste : retardement des opérations qui peuvent produire des problèmes de concurrence
- blocage des opérations en attente de verrous
 - ▶ verrou pour chaque enregistrement
 - ▶ chaque verrou est donné à une seule transaction à la fois
- en pratique: **verrou composé** = sous-verrou de lecture (partageable) + sous-verrou d'écriture (exclusif)

Algorithme de base

- 1) L'ordonnanceur reçoit $p_i[x]$ et teste les sous-verrous de x
 - ▶ si l'un des sous-verrous de x est détenu par une opération en conflit avec $p_i[x]$, alors $p_i[x]$ est retardée
 - ▶ sinon, verrou accordé à $p_i[x]$ et envoi $p_i[x]$ au GD
- 2) Un verrou pour $p_i[x]$ n'est jamais relâché avant la confirmation de l'exécution par le GD
- 3) Une fois un verrou pour T_i relâché, T_i n'obtiendra plus aucun verrou

Remarques

- règle 3 $\Rightarrow$ 2 phases: accumulation et relâchement de verrous

accumulation



relâchement

- règle 3 $\Rightarrow$ les paires d'opérations conflictuelles de T_i et T_j s'exécutent toujours dans le même ordre
- règle 2 $\Rightarrow$ l'ordre du GD pour les opérations conflictuelles sur un enregistrement x est le même que celui de l'ordonnanceur
- la variante la plus utilisée: relâchement de tous les verrous d'une transaction juste après *Commit*
 - ▶ assure le respect des règles 2 et 3
 - ▶ on peut démontrer que cela produit *des exécutions strictes*

Exemples

a) Non-respect de la règle de relâchement des verrous

$T_1: r_1[x] \ w_1[y] \ c_1$ $T_2: r_2[y] \ w_2[y] \ c_2$

ordre de réception: $r_1[x] \ r_2[y] \ w_1[y] \ c_1 \ w_2[y] \ c_2$

$H_1: r_1[x] \ r_2[y] \ (ru_2[y]) \ w_1[y] \ c_1 \ (ru_1[x] \ wu_1[y]) \ w_2[y] \ c_2 \ (wu_2[y])$

- notation: $ru_i[x]/wu_i[x]$ relâchement du verrou de lecture/écriture pour x par T_i
- violation règle: $ru_2[y]$ suivi de demande de verrou pour $w_2[y]$
- $r_2[y] - w_1[y], w_1[y] - w_2[y] \Rightarrow H_1$ non-sérialisable

b) Exécution correcte

$H_2: r_1[x] \ r_2[y] \ w_2[y] \ c_2 \ (ru_2[y] \ wu_2[y]) \ w_1[y] \ c_1 \ (ru_1[x] \ wu_1[y])$

- $w_1[y]$ retardée en attente du verrou sur y
- $r_2[y] - w_1[y], w_2[y] - w_1[y] \Rightarrow H_2$ sérialisable

c) Exécution sérialisable modifiée par le verrouillage

H: $r_2[y] \ r_1[x] \ w_2[x] \ c_2 \ w_1[z] \ c_1$

- conflits: $r_1[x] - w_2[x] \Rightarrow H$ sérialisable, équivalente à $T_1 T_2$
- $w_2[x]$ est bloquée, T_1 ne peut pas relâcher le verrou de lecture sur x , car elle aura besoin d'un autre pour $w_1[z]$
- Conclusion: certaines exécutions sérialisables ne sont pas acceptées telles quelles par le verrouillage à deux phases
 - ▶ le verrouillage ne profite pas de la sérialisabilité préalable des exécutions
 - ▶ cause: algorithme pessimiste

Interblocage

Exemple

T_1 : $r_1[x]$ $w_1[y]$ c_1

T_2 : $w_2[y]$ $w_2[x]$ c_2

ordre de réception: $r_1[x]$ $w_2[y]$ $w_2[x]$ $w_1[y]$

- T_1 obtient verrou pour $r_1[x]$, T_2 pour $w_2[y]$
- $w_2[x]$ attend $r_1[x]$, $w_1[y]$ attend $w_2[y]$
 $\Rightarrow$ interblocage de T_1 et de T_2

Stratégies pour éviter l'interblocage

- durée limite ("timeout")
 - ▶ rejet transaction non-terminée après une durée limite
 - ▶ problème: risque de rejet des transactions non-bloquées
 - ▶ paramétrage fin nécessaire pour la durée limite
- graphe d'attente
 - ▶ noeuds=transactions, arcs=attente de verrou
 - ▶ détection des cycles
 - ▶ annulation de la transaction la moins coûteuse
- risque: victime relancée et bloquée à nouveau; problème d'équité à l'annulation

3. Verrouillage hiérarchique

Données à plusieurs niveaux de granularité

- attribut < enregistrement < relation < base de données
- chaque opération se fait au niveau approprié
- verrouillage classique: un seul niveau de granularité
 - ▶ niveau élevé: gestion allégée, car moins de verrous
 - ▶ niveau bas: moins de conflits, plus de concurrence

Principes du verrouillage hiérarchique

- **verrouillage descendant:** implicite (par inclusion)
 - Ex. un verrou sur la relation est valable aussi pour chaque enregistrement, attribut, ... de la relation
- **verrouillage ascendant:** pour réaliser une opération à un niveau, on demande un *verrou d'intention* à tous les niveaux supérieurs
 - Ex. pour réaliser une écriture sur un enregistrement, on demande un verrou d'intention d'écriture sur la relation et la BD
 - ▶ obtention des verrous d'intention: descendante
 - ▶ relâchement des verrous d'intention: ascendant
- **verrouillage en escalade:** technique de choix du niveau de granularité
 - ▶ on commence par verrouiller au niveau fin (ex. enregistrement)
 - ▶ à partir d'un seuil, on passe au niveau de granularité supérieur (ex. relation)

Types de verrous

- **S** (partagé, lecture), **X** (exclusif, écriture)
- **IS** (intention lecture), **IX** (intention écriture)
- **SIX** (lecture et intention d'écriture)
 - ▶ cas très fréquent: lecture relation pour modifier quelques enregistrements
 - ▶ un seul verrou qui combine les verrous **S** et **IX**

Matrice de conflits entre verrous hiérarchiques

	X	SIX	IX	S	IS
X	o	o	o	o	o
SIX	o	o	o	o	n
IX	o	o	n	o	n
S	o	o	o	n	n
IS	o	n	n	n	n

Exemple

- données: relation **Compte**(*Numéro*, *Titulaire*, *Solde*)
- niveaux: relation, enregistrement
- T_1 : crédit du compte numéro 7 (lecture-écriture enregistrement x)
 T_2 : lecture de tous les comptes (lecture relation R)
 T_3 : initialisation compte 3 (écriture enregistrement y)

Exécution par verrouillage hiérarchique

- ordre de réception: $r_1[x]$ $r_2[R]$ $w_3[y]$ $w_1[x]$ c_1 c_2 c_3
 - $r_1[x]$: obtention $IS(R)$ et ensuite $S(x)$
 - $r_2[R]$: obtention $S(R)$ - pas de conflit avec $IS(R)$
 - $w_3[y]$: ne peut pas obtenir $IX(R)$ à cause de $S(R)$ → bloquée
 - $w_1[x]$: ne peut pas obtenir $IX(R)$ à cause de $S(R)$ → bloquée
 - c_1 bloquée car $T_1(w_1[x])$ bloquée
 - c_2 s'exécute, $S(R)$ relâché → opérations débloquées:
 - $w_3[y]$ obtient $IX(R)$ (pas de conflit avec $IS(R)$), puis $X(y)$
 - $w_1[x]$ obtient $IX(R)$ (pas de conflit avec $IS(R)$, $IX(R)$), puis $X(x)$
 - c_1 s'exécute
 - c_3 s'exécute

4. Création/destruction d'objets

Problème des objets fantômes

- contrôle de concurrence dans les BD dynamiques
- cas typique: T_1 consulte tous les objets d'un certain type, T_2 crée un objet de ce type

Exemple

- Relations **Compte**(*Numéro*, *Titulaire*, *Solde*) de comptes et **Total**(*Titulaire*, *Cumul*) de totaux par titulaire
- T_1 : compare la somme des comptes de "Dupont" avec le total pour "Dupont"
- T_2 : ajoute un compte pour "Dupont" et met à jour le total

$Read_1(Compte[5],Compte[8],Compte[14])$	100,300,600
$Insert_2(Compte[10,"Dupont", 500])$	
$Read_2(Total["Dupont"])$	1000
$Write_2(Total["Dupont"])$	1500
$Read_1(Total["Dupont"])$	1500

Compatible avec le verrouillage à 2 phases, mais non-sérialisable!

Solution

- pour consulter tous les objets d'un type: *info de contrôle* pour cet ensemble d'objets
- *verrouillage au niveau de l'info de contrôle*
- on traite l'ensemble comme un objet à part
 - ▶ parcours = lecture ensemble
 - ▶ insertion/suppression = écriture ensemble

Verrouillage d'index

- ensemble d'objets qui respectent *attribut* = *valeur* regroupés dans une entrée d'index
- entrée d'index sur *attribut*: *valeur* + liste pointeurs vers les objets
- verrouillage entrée d'index
 - ▶ parcourir les objets → lecture index
 - ▶ insérer un objet → écriture index

Verrouillage hiérarchique

- ensemble = niveau de granularité supérieur
- verrous sur la donnée de niveau supérieur
 - ▶ parcourir les objets → lecture niveau supérieur
 - ▶ insérer un objet → écriture niveau supérieur

Plan du cours

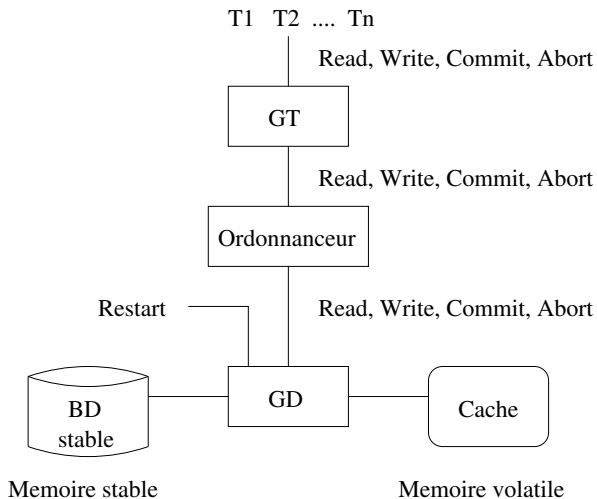
1 Concurrence et reprise après panne

- Introduction et problématique
- Contrôle de concurrence
- **Tolérance aux pannes**
- Transactions dans les SGBD relationnels

1. Problématique

Hypothèses

- pannes de système: contenu de la mémoire volatile perdu
 - ▶ opération *Restart* qui reconstitue un état cohérent de la BD
- ordonnanceur avec exécutions sérialisables et *strictes*
 - ▶ stricte: écritures validées dans l'ordre de Commit
- même granularité pour l'ordonnanceur et le GD
 - ▶ enregistrement = page disque



Objectif

- dernière valeur validée de x: dernière valeur écrite en x par une transaction validée
- état validé de la BD: l'ensemble des dernières valeurs validées pour tous les enregistrements
- panne: mémoire volatile perdue

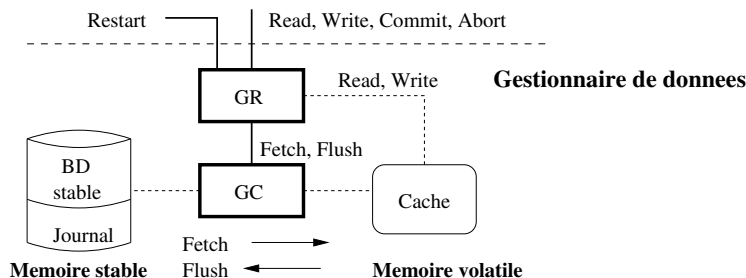
⇒ Restart doit ramener la BD à l'état validé avant la panne

- problèmes
 - ▶ annuler l'effet des transactions non-validées
 - ▶ terminer les transactions validées
 - ▶ structures à garder en mémoire stable pour assurer la reprise

2. Architecture

Les composantes du Gestionnaire de données (GD)

- **Gestionnaire du Cache (GC):** gère les deux mémoires
- **Gestionnaire de reprise (GR):** opérations BD + Restart



Gestionnaire du Cache

- utilisation de la mémoire volatile : rapidité
- idéal: copie de toute la BD
- en réalité: caching, car taille mémoire volatile limitée

Cache

- zone de mémoire volatile divisée en *cellules*: 1 enreg./cellule
- en réalité, le Cache stocke des *pages disque*

Cache

nr. cel.	bit de consistance	valeur enreg.
1	1	"J. Smith"
2	0	2.24581
.....		

Repertoire du cache

id. enreg.	nr. cel.
x	2
y	1
.....	

Opérations

- Flush (c), c cellule

si c inconsistante alors

copier c sur disque

rendre c consistante

sinon rien;

- Fetch (x), x enregistrement (pas dans le Cache)

sélectionner c cellule vide

si toutes les cellules occupées alors

vider une cellule c avec **Flush** et l'utiliser comme cellule vide

copier x du disque en c

rendre c consistante

mettre à jour le répertoire du cache avec (x, c)

- choix cellule vide: LRU, FIFO
- lecture x :
 - ▶ toujours à partir du cache
 - ▶ si x n'est pas dans le cache alors Fetch(x) d'abord
- écriture x :
 - ▶ soit c la cellule de x dans le Cache (allouée à ce moment-là si x n'y est pas déjà)
 - ▶ c modifiée, marquée inconsistante
 - ▶ Flush(c) décidé par GR, selon son algorithme

Gestionnaire de reprise

Opérations

- GR_Read (T_i, x)
- GR_Write (T_i, x, v)
- GR_Commit (T_i)
- GR_Abort (T_i)
- Restart

3. Journalisation

Journal

- historique des écritures dans la mémoire stable
- **journal physique**: liste de $[T_i, x, v]$
 - ▶ préserve l'ordre des écritures: fichier séquentiel
 - ▶ souvent on stocke aussi *l'image avant* de l'écriture
- **journal logique**: opérations de plus haut niveau
 - Ex. insertion x dans R et mise-à-jour index
 - ▶ moins d'entrées, mais plus difficile à interpréter
- autres informations: listes de transactions actives, validées, annulées

Ramasse-miettes

- recyclage de l'espace utilisé par le journal
- règle:
 - entrée $[T_i, x, v]$ recyclée $\Leftrightarrow$
 - T_i annulée ou
 - T_i validée, mais une autre T_j validée a écrit x après T_i

4. Principes et techniques pour la reprise

Types de GR

- GR peut forcer ou non GC d'écrire des cellules du Cache sur disque
- GR lance l'annulation
 - ▶ permet aux transactions non-validées d'écrire sur disque
 - ▶ *Restart* doit annuler ces écritures (annulation)
- GR lance la répétition
 - ▶ permet aux transactions de valider avant d'écrire sur disque
 - ▶ *Restart* doit refaire ces écritures (répétition)
- 4 catégories de GR (combinaisons annulation - répétition)

Règles défaire/refaire

- règles de journalisation, nécessaires pour que le GR puisse faire correctement annulation/répétition
- Règle “défaire” (pour annulation): si x sur disque contient une valeur validée, celle-ci doit être journalisée avant d’être modifiée par une valeur non-validée
- Règle “refaire” (pour répétition): les écritures d’une transaction doivent être journalisées avant son Commit
- *Remarque*: ces règles sont naturellement respectées si l’on écrit dans le journal avant toute écriture dans la BD

Idempotence de Restart

- une panne peut interrompre toute opération, même *Restart*
- idempotence: *Restart* interrompu et relancé donne le même résultat que le *Restart* complet
- optimisation: journalisation des opérations de *Restart* pour ne pas tout recommencer

Checkpointing

- ajouter des informations sur disque en fonctionnement normal afin de réduire le travail de *Restart*
- *point de contrôle* (“checkpoint”): point (marqué dans le journal) où l’on réalise les actions supplémentaires

Quelques techniques

- marquer dans le journal les écritures déjà **réalisées**/annulées dans la BD stable
 - ▶ pas besoin de refaire/annuler ces écritures à la reprise
- marquer toutes les écritures **validées**/annulées dans la BD stable
 - ▶ pas besoin de refaire/annuler à la reprise les transactions validées/annulées

5. Algorithme annulation/répétition

Principes

- GR qui demande annulation et répétition: le plus complexe
- écrit les valeurs dans le Cache et ne demande pas de Flush
- avantages: flexibilité, minimise I/O

Opérations

- GR-Write (T_i, x, v)

liste_active = **liste_active** $\cup \{T_i\}$

si x n'est pas dans le cache alors allouer cellule pour x

journal = **journal** + [T_i, x, v]

cellule(x) = v

confirmer Write à l'ordonnanceur

- GR-Read (T_i, x)

si x n'est pas dans le cache alors **Fetch**(x)

retourner la valeur de *cellule*(x) à l'ordonnanceur

- GR-Commit (T_i)

liste_commit = **liste_commit** $\cup$ $\{T_i\}$

confirmer le Commit à l'ordonnanceur

liste_active = **liste_active** - T_i

- GR-Abort (T_i)

pour chaque x écrit par T_i

si x n'est pas dans le cache alors allouer cellule pour x

cellule(x) = **image_avant**(x , T_i)

liste_abort = **liste_abort** $\cup$ $\{T_i\}$

confirmer Abort à l'ordonnanceur

liste_active = **liste_active** - $\{T_i\}$

- Restart

marquer toutes les cellules comme vides

refait = {}, **annulé** = {}

pour chaque $[T_i, x, v] \in \text{journal}$ (à partir de la fin) où $x \notin \text{annulé} \cup \text{refait}$

si x n'est pas dans le cache alors allouer cellule pour x

si $T_i \in \text{liste_commit}$ alors

$\text{cellule}(x) = v$

refait = **refait** $\cup \{x\}$

sinon

$\text{cellule}(x) = \text{image_avant}(x, T_i)$

annulé = **annulé** $\cup \{x\}$

si **refait** $\cup$ **annulé** = BD alors stop boucle

pour chaque $T_i \in \text{list_commit}$

list_active = **list_active** - $\{T_i\}$

confirmer Restart à l'ordonnanceur

6. Autres algorithmes

Algorithme annulation/sans-répétition

- GR ne demande jamais répétition
- enregistre écritures avant le Commit
- GR-Write, GR-Read, GR-Abort pareil
- GR-Commit pareil, mais d'abord:
 - ▶ pour chaque x écrit par T_i , si $x \in \text{Cache}$ alors $\text{Flush}(x)$
- Restart pareil, sauf que "refait" n'existe pas

Algorithme sans-annulation/répétition

- GR ne demande jamais annulation
- écritures des T_i non-validées retardées après Commit
- GR-Write: ajoute juste $[T_i, x, v]$ au journal
- GR-Read: si T_i a déjà écrit x , lecture dans le journal, sinon dans la BD
 - ▶ si T écrit x , les autres transactions ne peuvent lire x qu'après la fin de T (exécution stricte)
- GR-Commit: chaque x écrit par T_i est calculé à partir du journal et écrit dans le cache
- GR-Abort: juste ajoute T_i à liste_abort
- Restart: pareil, sauf que "annulé" n'existe pas

Algorithme sans-annulation/sans-répétition: les écritures de T_i réalisées sur disque en une seule opération atomique, au Commit

Plan du cours

- 1 Concurrence et reprise après panne
 - Introduction et problématique
 - Contrôle de concurrence
 - Tolérance aux pannes
 - Transactions dans les SGBD relationnels

Contrôle de concurrence

- basé sur le verrouillage à deux phases hiérarchique
- validation = COMMIT, annulation = ROLLBACK
- la norme ne prévoit pas le verrouillage *explicite* au niveau programmation (SQL): *4 niveaux d'isolation*
- transactions: deux caractéristiques importantes
 - ▶ le niveau d'isolation: SERIALIZABLE, REPEATABLE READ, READ COMMITTED, READ UNCOMMITTED
 - ▶ le mode d'accès: READ ONLY, READ WRITE
- commande: SET TRANSACTION *niveau accès*

Niveaux d'isolation

- ① **SERIALIZABLE**: le seul niveau qui garantit la sérialisabilité!
 - ▶ isolation totale et protection contre les objets fantômes
- ② **REPEATABLE READ** (lecture répétable)
 - ▶ les données lues par la transaction ne sont pas modifiables par d'autres transactions
 - ▶ ne **lit** que des valeurs validées (pas de lecture sale)
 - ▶ moins strict que **SERIALIZABLE**, ne protège pas contre les objets fantômes
- ③ **READ COMMITTED** (lecture de valeurs validées)
 - ▶ ne **lit** que des valeurs validées (pas de lecture sale)
 - ▶ n'assure pas la lecture répétable, ne protège pas contre les objets fantômes
 - ▶ les verrous S sont relâchés immédiatement
- ④ **READ UNCOMMITTED** (lecture de valeurs non-validées)
 - ▶ permet les lectures sales, n'assure pas la lecture répétable, ne protège pas contre les objets fantômes
 - ▶ niveau **limité** aux transactions **READ ONLY**
 - ▶ ne demande pas de verrou S (lecture sale), ni X (read only)
 - ▶ annulations en cascade possibles

Particularités dans les systems réels

- *SQL Server*: utilise aussi l'estampillage
 - ▶ chaque transaction a une estampille ($T_i \rightarrow i$)
 - ▶ l'ordre des conflits doit respecter l'ordre des estampilles
 - ▶ optimiste: pas d'attente à un verrou
- *Oracle*: contrôle de concurrence *multi-version*
 - ▶ on maintient plusieurs versions de chaque donnée
 - ▶ ordonnancement de type estampillage
 - ▶ pour chaque version on connaît la transaction qui l'a écrite
 - ▶ pour chaque donnée on connaît la dernière transaction qui l'a lue
 - ▶ une lecture n'attend jamais!
 - ★ elle utilise la dernière version qui respecte l'ordre
 - ▶ une écriture crée une nouvelle version ou est annulée
 - ★ annulation si on ne peut pas respecter l'ordre

Reprise après panne

- respect des principes généraux, avec de nombreuses variantes
- journalisation physique avec points de contrôle (checkpointing)
- *point de sauvegarde* (savepoint): utile pour les transactions longues
 - ▶ *Savepoint s* = validation partielle d'une transaction
 - ▶ on peut faire une annulation partielle *Rollback to s*

Commit à deux phases

- validation de transactions réparties sur plusieurs sites
- chaque site gère ses propres ressources (données, journal)
- *première phase*: chaque site valide ses propres opérations
- *seconde phase*: le gestionnaire global valide l'ensemble de la transaction
- la validation globale $\rightarrow$ seulement si chaque site valide
- *Rollback* d'un site $\Rightarrow$ *Rollback* de l'ensemble
- le gestionnaire annonce chaque site si la transaction doit être validée ou annulée